



What a Dormant Agent Costs

State, Execution and
Isolation Measured Separately

at a Fleet of One Million

Hanzo AI Research

Hanzo AI

Measurement pass: 2026-09-07 (Apple M-series, darwin/arm64, 16 cores, Go 1.27.1)

What a Dormant Agent Costs

State, Execution and Isolation Measured Separately

at a Fleet of One Million

Hanzo AI Research*
Hanzo Industries Inc (Techstars '17)
Los Angeles, California
<https://hanzo.ai>

Measurement pass: 2026-09-07 (Apple M-series, darwin/arm64, 16 cores, Go 1.27.1)

Abstract

An agent platform is usually costed as though an agent were a process. It is the natural reading of the deployment shape — one container per agent — and it sets the per-agent floor at the container runtime’s minimum reservation, which on commodity platforms is 128–512 MB. At that floor a fleet of a million agents is a datacenter, and every design decision downstream of the estimate is made in its shadow.

We take the estimate apart. An idle agent is three separable things: the *state* it will resume from, the *execution context* that holds its loop, and the *isolation boundary* its code runs behind. These have no reason to cost the same, and measured on one laptop they differ by five orders of magnitude. State at rest is a row: **477 bytes** on disk [measured], so 1,000,000 dormant agents occupy **454.6 MiB** — written at 309,789 agents/s and resumed in **0.034 ms** in-process. Execution at rest is a goroutine: **601 bytes** of heap [measured], identical on every run, so a million live loops are 573.0 MB and each starts in 187 ns. Isolation is the expensive one, and it is expensive for a reason: an in-process V8 context costs 0.15 ms [measured] but runs only JavaScript, while an OCI container costs 150.80 ms — 1016× more — and is the only one of the two that can run a test suite, a package manager or a shell.

The conclusion is not that containers are wasteful. It is that a fleet pays for isolation *per running agent*, not per registered one, while the industry estimate charges the isolation price to every agent in the fleet whether or not it is doing anything. Separating the three costs makes the dormant majority nearly free: 1,000,000 dormant agents are 0.01 dollars a month of storage [modeled] beside a control plane that costs \$852 [cited]. We give the capacity model this implies, and we are explicit in §8 about what 601 bytes is: a floor for an agent holding nothing, not a forecast for an agent holding a conversation.

1 Introduction

Ask what it costs to run a million agents and the answer usually arrives as one number. That number is almost always the memory floor of a container, because the deployment shape that has become idiomatic for agent platforms is one container per agent, and a container on the commodity serverless platforms reserves 128 MB at the low end and 512 MB at the high [8, 7, 9]. A million of those is between 128 and 512 terabytes of resident memory, which is a datacenter, and a platform

*Corresponding author: research@hanzo.ai

that believes this about itself will build a scheduler, a billing model and a market position around a scarcity its own hardware does not impose.

The estimate is wrong because the unit is wrong. “An agent” is not one thing with one cost. It is three things that happen to be discussed together:

State what the agent will resume from — its identity, its position in a workflow, its handle on a conversation. This exists whether or not the agent is running, and it is data.

Execution

the context that holds the agent’s loop while it waits: a stack, a scheduler entry, a place for a continuation to live. This exists only while the agent is resident.

Isolation

the boundary the agent’s code runs behind, which is what stops agent *a* reading agent *b*’s filesystem or exhausting the host. This exists only while the agent’s code is actually executing.

Nothing requires these to be the same object, and when they are separated they turn out to differ by five orders of magnitude. This paper measures each one on the same laptop, at a fleet of a million, and reports what it found — including where the expensive primitive is worth its price.

The finding that motivates the rest: **an agent that is doing nothing does not need an execution context or an isolation boundary at all.** It needs a row. A row costs 477 bytes [measured] and can be brought back into a running loop in 0.034 ms [measured], which is fast enough that keeping the loop alive to avoid the resume is a false economy for any agent idle longer than a few milliseconds. The dormant majority of a fleet — which is most of any real fleet, most of the time — therefore costs storage rather than memory, and storage is four orders of magnitude cheaper.

This is the same argument, one level down, that the companion tenant-residency paper makes about databases [1]: there the unit was a whole per-tenant SQLite file at ≈ 0.25 MB of resident memory, and the result was that memory scales with concurrency rather than registrations. Here the unit is a single agent inside such a file, and the result has the same shape: cost tracks *activity*, not *population*. That the two hold at different granularities is the point — it is a property of the decomposition, not of a particular size.

Contributions

1. A separation of the per-agent cost into state, execution and isolation, with each measured independently rather than inferred from a deployment shape (§3–§6).
2. Measured residency for a fleet of 1,000,000 dormant agents: 477 bytes each on disk, written at 309,789/s, resumed in 0.034 ms in-process (§3).
3. Measured residency for 1,000,000 *live* execution contexts: 601 bytes of heap per goroutine, reproducible to the byte across five runs, with spawn and wake latencies in the hundreds of nanoseconds (§4).
4. Instantiation cost for three in-process language runtimes — WebAssembly, JavaScript and Python — all in the single-digit to tens of microseconds, and all without a container (§5).
5. A like-for-like cold-start comparison of the two isolation primitives that are usually conflated, with an argument for why the 1016 $\times$ ratio between them is not a defect (§6).

6. A capacity and cost model built only from the measured constants, and a statement of the conditions under which it stops holding (§7–§8).

2 Method

2.1 Host and provenance

Every measurement in this paper was taken on one machine: an Apple M-series laptop, `darwin/arm64`, 16 cores, Go 1.27.1, Node 26.5.0, on 2026-09-07. A laptop is a deliberate choice. The claim under test is that a fleet of a million agents does not require special hardware, and a result taken on special hardware could not support it.

Numbers are labelled throughout. `[measured]` was produced by a named script on that host. `[modeled]` was computed from measured constants and stated assumptions. `[cited]` came from a published source, with the source given. No number in this paper is any other kind.

Each measured constant reaches the text through a macro defined in `results-data.tex`, which is generated from the runs. If that file is absent the paper prints **XXX** rather than a stale literal, so a number cannot drift away from the run that produced it without the drift being visible on the page.

2.2 What each benchmark does

The four scripts live in `hanzoai/cloud` under `bench/` and each re-runs to reproduce its table.

`fleet/fleet.mjs`

writes 1,000,000 dormant agent records into SQLite through the production schema (100,000 tenants $\times$ 10 agents), then measures the file, the write rate, and the latency of resuming one agent both in-process and through a cold process spawn.

`goroutine/main.go`

spawns 1,000,000 goroutines, each parked on a channel as an agent loop would be, and measures spawn latency, heap growth against `runtime.MemStats` after a forced GC, and the time to wake all of them. It also instantiates the three in-process language runtimes.

`sandbox/sandbox.mjs`

measures cold start for an in-process V8 context, a fresh OCI container, and a command dispatched into an already-warm container, 20 runs each.

`pricing/pricing.mjs`

converts the measured volumes into a monthly bill at published cloud rates.

2.3 Repetition, and one method note that changed the numbers

Memory results are deterministic here and are reported as a single figure: the per-goroutine cost came out at exactly 601 bytes on all five runs. Latency results are not deterministic, and are reported as a median with the observed range, never as a best-of.

The one trap worth recording is that the first goroutine measurements were taken under `go run`, which compiles the program as part of the invocation. That inflated spawn latency by roughly a third (253 ns against a median of 187 ns from a built binary) and inflated the Python context figure by about 15%. *Benchmark a built artifact*. The rows published here come from `go build` output, run five times, serialized so that no two benchmarks contend for the same cores.

3 State at rest: an agent is a row

A dormant agent’s state is whatever it must be handed to carry on: an identifier, the tenant it belongs to, its position in a workflow, a pointer to its conversation, and whatever the agent itself has stashed. Serialized through the production schema this is 367 bytes of JSON for a representative agent [measured].

We wrote 1,000,000 of them — 100,000 tenants with 10 agents each — into SQLite and measured the file.

Fleet of 1,000,000 dormant agents	
Agent record, serialized	367 bytes [measured]
On disk, per agent	477 bytes [measured]
On disk, total	454.6 MiB [measured]
Write time	3.2 s [measured]
Write rate	309,789 agents/s [measured]
Resume, in-process	0.034 ms [measured]
Resume, via process spawn	3.42 ms [measured]

Table 1: State residency. The gap between the serialized record and the on-disk figure is SQLite’s per-row and index overhead: 477 against 367 is 110 bytes of structure per agent, which buys the index that makes the resume in the last row possible.

Three things in that table matter more than the headline.

The whole fleet fits in a file you can copy. 454.6 MiB is smaller than a single container image. It fits in page cache on the laptop it was measured on. There is no storage tier involved and no distributed system: it is one file, and the standard tools work on it. This is what makes the dormant majority cheap — not a compression scheme or a tiering policy, but the absence of anything that would make it expensive.

Writing is not the bottleneck anyone expects. 309,789 agents/s means the entire million-agent fleet is created in 3.2 seconds on one core of a laptop. A platform provisioning agents at even a thousand a second — which would be a very busy platform — is using 0.3% of the write path measured here.

Resume is cheap enough to prefer over residency. 0.034 ms in-process is the number that settles the design question. If bringing an agent back from a row costs 34 microseconds, then holding its execution context alive to avoid that cost is only worth it for an agent that will be woken again within microseconds. For anything idle longer — which is what dormant means — the row is strictly better: it costs 477 bytes of disk instead of 601 bytes of RAM, and it survives a restart.

The contrast in the last row is worth stating plainly. Resuming through a cold process spawn costs 3.42 ms, which is 100× the in-process figure. That entire 100× is process creation, not agent work. It is the first appearance in this paper of the pattern that dominates the rest of it: *the cost is almost never the agent, it is the container the agent was put in.*

4 Execution at rest: an agent is a goroutine

An agent that *is* running — waiting on a tool call, a model response, or another agent — needs something to hold its place. In the shape this platform uses that something is a goroutine: a stack, a scheduler entry, and a channel it is parked on. Go allocates a goroutine an initial stack of 2 KiB that grows and shrinks on demand [2], so a fleet’s memory is bounded by what the agents actually hold rather than by a per-agent reservation chosen in advance.

We spawned 1,000,000 goroutines, each parked as an agent loop would be, and measured heap after a forced garbage collection.

1,000,000 live execution contexts	median	range
Spawn, per agent	187 ns	174–216 ns [measured]
Heap, per agent	601 bytes	identical, 5 runs [measured]
Heap, total	573.0 MB	identical, 5 runs [measured]
Wake all, per agent	114 ns	112–120 ns [measured]

Table 2: Execution residency, five runs of a built binary. Memory was byte-identical on every run; latencies are medians with the observed range.

601 bytes, not 2 KiB. The initial stack is 2 KiB, but the measured cost is 601 bytes, because a parked goroutine that has not descended into a deep call has not touched most of its stack and the runtime has not committed it. The figure was identical on all five runs, which is what one expects from a deterministic allocator under a forced GC and is a useful check that the measurement is measuring anything at all.

Against the container floor this is the paper’s widest ratio. A commodity serverless container reserves 128 MiB at minimum [8]; a parked agent loop costs 601 bytes. That is a factor of roughly 223,000. The two are not substitutes — §6 is about exactly what the container buys — but a great many agents in a real fleet are waiting on a network round trip and need none of it.

Spawn and wake are effectively free. At 187 ns, creating an agent’s execution context is cheaper than a single cache miss. At 114 ns per agent, waking the entire million-agent fleet takes 114 ms. Neither number constrains any design: an architecture that avoids spawning agent contexts to save time is optimizing something that costs nanoseconds.

What this measures, and what it does not. It measures an *empty* loop — an agent holding a place, not a conversation. A real agent holds context, and that context is the dominant term in its live memory. 601 bytes is therefore a *floor* on execution residency and not a prediction of it. §8 takes this seriously, because it is the single easiest number in this paper to misuse.

5 Code: three language runtimes, none of them a container

An agent that only waits is not interesting. At some point it runs code — a tool implementation, a transformation, a policy the tenant wrote. The reflex is that running tenant code requires a container, and §6 is about when that reflex is right. But there is a tier between “call a Go function” and “start Linux”, and it is not exotic: an interpreter, in the host process, with the host deciding what it can reach.

We measured three, all pure Go, all linked into the same binary that holds the agent loops.

Runtime	Runs	Instantiate	Range
wazero [3]	WebAssembly	8.9 μ s	8.9–9.5 μ s [measured]
goja [4]	JavaScript (ES5.1+)	2.7 μ s	2.6–2.8 μ s [measured]
gpython [5]	Python	29.9 μ s	29.1–30.8 μ s [measured]

Table 3: Cost of a fresh execution context for tenant code, in-process. Calling into an instantiated wazero module costs 23 ns; re-evaluating in an already-warm goja VM costs 818 ns (812–829 ns).

All three are microseconds. The slowest, Python, is 29.9 μ s — still four orders of magnitude below a container cold start, and roughly the cost of a single disk seek on hardware that has not had one in a decade.

WebAssembly is not one language. The wazero row deserves emphasis because it is easy to file under “Rust and Go”. WebAssembly is a compilation target, and CPython, TypeScript via QuickJS, Rust, Go, C and C++ all reach it. A 8.9 μ s instantiation is therefore not a JavaScript-shaped capability; it is a general one, with a capability model that is deny-by-default — a module reaches exactly the host functions it was linked against and nothing else [6]. That is a stronger default than a container, which starts with a filesystem and a network stack and must have them taken away.

Warm is a different number, and usually the relevant one. 818 ns to evaluate in an existing VM against 2.7 μ s to make a new one is a factor of about three thousand. A fleet that creates one context per call is paying the left column; a fleet that keeps a context per active agent pays the right. Since §3 showed that agent state is a row that can be rehydrated in 0.034 ms, the natural arrangement is to key a pooled context by agent and let it fall away when the agent goes dormant — which converts the instantiation cost from per-call to per-wake.

The honest limit. None of these is a security boundary against an actively hostile tenant in the way a virtual machine is. goja and gpython are interpreters in the host address space: a bug in one of them is a bug in the host process. wazero is stronger — the sandbox is the WebAssembly semantics themselves, and the module cannot name what it was not given — but it is still one process. Where the threat model includes an adversary who will try to escape, the answer is the next section, and it costs a thousand times more for exactly that reason.

6 Isolation: what a boundary costs, and what it buys

Sandbox comparisons are usually published as one column of milliseconds, and that column conflates two primitives that are not alternatives. We measured both on the same host, 20 runs each.

The ratio is real and it is not a defect. 1016 $\times$ is what a kernel-enforced boundary costs: namespaces, a mount table, a network interface, a cgroup, and an image to unpack. A V8 context is a data structure in a process that is already running, so it starts in 0.15 ms and offers a boundary that is exactly as strong as the embedding process’s correctness. These buy different things and the choice is per workload, not per millisecond.

Primitive	median	p95	What it can run
V8 context, in-process	0.15 ms	0.19 ms	JavaScript. No shell, no pip, no pytest.
OCI container, cold	150.80 ms	188.80 ms	Any language, any binary, a real filesystem.
OCI container, pooled	37.54 ms	47.55 ms	As above, on a container already started.

Table 4: Cold start by isolation primitive, 20 runs each. The measured ratio between a cold container and an in-process V8 context is 1016 $\times$.

The workload decides it, and for agents the workload is frequently unambiguous. An agent asked to run a project’s test suite needs a package manager, a filesystem and a shell. No JavaScript isolate provides those at any latency. The right comparison for that workload is the second row against other people’s second rows — and 150.80 ms compares favourably to the published cold-start figures for hosted sandbox platforms, which range from under 200 ms to several seconds [7, 8, 9].

Pooling collapses most of the gap, and is the only tier that matters at fleet scale. 37.54 ms is a command dispatched into a container that is already up. It is 4 $\times$ better than a cold start and it is what a fleet actually experiences, because a fleet reuses sandboxes rather than creating one per call. The cold number is the right one to quote for a first call after a scale-up; the pooled number is the right one for steady state, and quoting the cold number as the platform’s latency is as misleading as quoting the pooled one as its worst case.

Three tiers, chosen by threat model. Taken with §5, the measurements describe a ladder, and the sensible policy is to take the cheapest rung the threat model permits:

1. **In-process call** — our own code, no boundary, nanoseconds.
2. **In-process runtime** — tenant code that is not adversarial, a capability boundary, 8.9–29.9 μ s (§5).
3. **Container** — arbitrary binaries, an adversarial tenant, or a real filesystem: 37.54 ms pooled, 150.80 ms cold.

A platform that offers only the third rung charges every workload the price of the most demanding one. That is the cost this paper is trying to make visible — not the container itself, which is worth its price when it is needed.

7 Capacity, from the measured constants only

The constants in §3–§6 determine a capacity model. Everything in this section is [modeled] — arithmetic over measured inputs and stated assumptions — and nothing in it was observed at the scales named. We give it because the arithmetic is the point of the decomposition, and we mark it so that no reader mistakes it for a result.

7.1 Storage is set by population, and is negligible

Dormant agents are 477 bytes each, so storage is exactly linear in how many agents exist:

Registered agents	On disk	At list price/mo
10^6	454.6 MiB	\$0.01 [measured]/[modeled]
10^8	44.4 GiB	\$0.89 [modeled]
10^9	444.2 GiB	\$8.90 [modeled]

A billion registered agents is under half a terabyte and under ten dollars a month of object storage. Storage is not the constraint and it will not become the constraint; §8 names the thing that does.

7.2 Memory is set by concurrency, and the working set dominates

Live agents cost memory. The measured floor is 601 bytes, but a working agent holds context, so the honest model is parameterized by that working set rather than by the floor. On a 128 GiB node:

Per-agent working set	Live agents/node	10^9 reg. @ 1%	@ 0.1%
601 B (measured floor)	228 M	1 node	1 node
10 KiB	13.4 M	1 node	1 node
64 KiB	2.1 M	5 nodes	1 node
1 MiB	131 k	77 nodes	8 nodes

[modeled] throughout. The left column is the one that matters and it is the one this paper did not measure: a fleet’s cost is set by how much context its agents hold while running, not by the residency floor of the primitive holding them. What the measurement establishes is that the *floor* contributes nothing — at 601 bytes it is invisible against any realistic working set — so the platform’s overhead is not the thing to optimize.

The structural claim survives the parameterization: memory tracks the agents that are *awake*. Under any of these rows, a billion registered agents at realistic concurrency is a handful of commodity nodes rather than a datacenter, and the difference between this and the 128 TB estimate in §1 is entirely that the estimate charged every registered agent for an isolation boundary it was not using.

7.3 What the bill looks like

At published rates for the always-on control plane — 14 vCPU and 56 GB, \$852/month [10] [cited] — the fixed cost of the platform exceeds the storage cost of a billion dormant agents by two orders of magnitude. In other words, at any population, *the agents are not the bill*. The bill is the compute for the agents that are awake, plus the item in §8 that grows without bound.

We state one exclusion emphatically, because omitting it would make every figure here misleading: **model tokens are not in this model and they dominate it**. A single frontier-model call costs cents; the infrastructure under an agent turn costs microdollars. Any per-call price derived from this paper sits *beside* token billing and must never be presented as including it.

8 Limits

601 bytes is a floor and will be quoted as a forecast. This is the most likely misuse of the paper. The measured goroutine holds a place and nothing else. A working agent holds a

conversation, a tool schema, and whatever it has retrieved — kilobytes to megabytes — and that term dominates 601 bytes by three to four orders of magnitude. The correct reading is narrow and still useful: *the platform’s own per-agent overhead is negligible*, so a fleet’s memory is the agents’ data and not the runtime’s bookkeeping. Any capacity claim that multiplies 601 bytes by a population is wrong, including if we make it.

One host, one architecture. Every number is from one Apple M-series laptop. The goroutine and SQLite results should port to any 64-bit platform with the same Go and SQLite versions — they measure allocator and file-format behaviour, not silicon — but the container figures will not: they were taken through a Linux VM on macOS, which adds a virtualization layer that a native Linux host does not have. 150.80 ms is therefore likely *pessimistic* for our own cold-start row. We have not measured it natively and do not claim the better number.

History is the term that grows. A dormant agent stays 477 bytes forever. Its transcript does not. At a retained 2 KiB per call, a fleet serving a billion calls a month accumulates roughly 22 TB over twelve months [modeled] — which eventually exceeds compute, and is the one line in the model with no ceiling. A platform built on this decomposition must have a retention policy before it needs one; the cheap dormant agent is what makes the expensive transcript easy to overlook.

Resume assumes the state is reachable. 0.034 ms is an in-process read from a local file. An agent whose state lives on another node pays a network round trip instead, and the design that makes this rare — pinning an agent’s tenant to a writer, as the companion paper describes [1] — has its own failure modes on failover. The measurement is of the storage layer, not of a distributed system.

The in-process runtimes are not a boundary against a hostile tenant. Stated in §5 and repeated because it is the one place where taking the cheap rung would be a security error rather than an economy: goja and gpython run in the host address space. Where the tenant is adversarial, the third rung is the only correct one, and its price is the price.

What we did not measure. Scheduler behaviour under contention at a million runnable (rather than parked) goroutines; garbage-collection pause time at that heap; SQLite behaviour under concurrent writers across many tenants; and the end-to-end latency of a real agent turn, which is dominated by the model call and would tell you nothing about the primitives this paper is about.

9 Related work

Hosted sandbox platforms publish cold-start and minimum-memory figures for the container tier: E2B under 200 ms with a 512 MB minimum [7], Modal around a second with a 128 MB minimum [8], and Cloudflare Workers 1–3 s for a container with 256 MB [9]. These are the numbers that set the industry estimate this paper takes apart, and §6 measures the same primitive rather than a cheaper one in order to compare honestly.

The in-process tier has a long lineage that agent platforms have mostly not drawn on. V8 isolates as a multi-tenant unit are the basis of edge function platforms [9]; WebAssembly’s capability model gives a deny-by-default boundary at a fraction of a container’s cost [6]; and pure-Go implementations of all three runtimes measured here [3, 4, 5] make the tier available without CGO or a second process. The contribution of §5 is not the runtimes but the measurement that all three

sit in the microseconds, which is what makes the three-rung ladder in §6 a practical policy rather than a taxonomy.

The residency argument is the agent-scale case of the tenant-scale one made in [1], where a per-tenant SQLite file was measured at ≈ 0.25 MB resident and the conclusion was that memory scales with concurrency rather than registrations. Green threads as a cheap concurrency unit predate all of this and the goroutine implementation’s growable stacks [2] are what make the 601-byte figure possible.

10 Conclusion

An agent is not a process, and costing it as one inflates the estimate by roughly five orders of magnitude. Measured separately on one laptop: state at rest is 477 bytes, execution at rest is 601 bytes, and isolation is 0.15–150.80 ms depending on which boundary the workload actually needs.

The practical consequence is a policy rather than a product. Take the cheapest rung the threat model permits; keep dormant agents as rows and rehydrate them in 0.034 ms rather than holding loops open; pool the expensive boundary so that a fleet pays 37.54 ms in steady state instead of 150.80 ms per call; and size memory against what the agents *hold*, since the primitives holding them contribute nothing measurable.

The result we would most like to see contested is the one we flagged hardest: 601 bytes is the residency of an agent that holds nothing, and a fleet of agents that hold something will be dominated by that something. The measurement says the platform overhead is not the problem. It does not say there is no problem.

Reproducing

The scripts are in `hanzoai/cloud` under `bench/`, one directory per measurement, and each prints the table it produced:

```
node fleet/fleet.mjs /tmp/fleet      # state at rest
go build -o /tmp/goro . && /tmp/goro  # execution at rest, and the runtimes
node sandbox/sandbox.mjs              # isolation (needs a container runtime)
node pricing/pricing.mjs              # rates
```

References

- [1] Hanzo AI Research. The Unified Sovereign-Tenant Cloud: Linear Shared-Nothing Scaling via Per-Tenant SQLite and In-Process Composition. Hanzo AI Research, 2026.
- [2] The Go Authors. Goroutines and the runtime scheduler: growable stacks. <https://go.dev/doc/>, Go 1.27 runtime documentation.
- [3] Tetrate. wazero: the zero-dependency WebAssembly runtime for Go. <https://wazero.io>.
- [4] D. Siegel et al. goja: ECMAScript/JavaScript engine in pure Go. <https://github.com/dop251/goja>.
- [5] N. Craig-Wood. gpython: a Python interpreter written in Go. <https://github.com/go-python/gpython>.

- [6] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, and J. F. Bastien. Bringing the Web up to Speed with WebAssembly. In *PLDI*, 2017.
- [7] E2B. Sandbox documentation: startup time and resource limits. <https://e2b.dev/docs>.
- [8] Modal. Documentation: cold start and container resource minimums. <https://modal.com/docs>.
- [9] Cloudflare. Workers and Containers: isolates, startup latency and instance types. <https://developers.cloudflare.com/workers/>.
- [10] Railway. Pricing: vCPU and memory rates. <https://railway.com/pricing>.